

Урок 9. JS — методы массивов

Массив — хранилище для упорядоченного (индексированного) набора данных.

Методы массива — это встроенные функции, которые позволяют управлять массивом и манипулировать его содержимым. Их делят на две группы:

- методы, которые изменяют исходный массив (например `push()`) — их называют мутлирующими;
- методы, которые возвращают новый массив, не трогая исходный.

Методы, изменяющие массив (мутлирующие)

Эти методы меняют исходный массив «на месте». В Angular такие методы стараются не использовать — фреймворк построен вокруг иммутабельности данных (неизменяемости), поэтому вместо мутации обычно создают новый массив.

- `push()` — добавляет новый элемент в конец массива.
- `pop()` — удаляет последний элемент массива.
- `unshift()` — добавляет элемент в начало массива.
- `shift()` — удаляет первый (начальный) элемент массива.
- `reverse()` — переворачивает массив: последний элемент становится первым и наоборот.

Методы, возвращающие новый массив

Эти методы не трогают исходный массив — они всегда создают новый.

`map()`

Метод `map()` внутри себя выполняет колбэк — функцию, переданную аргументом в другую функцию, часто анонимную (без собственного имени). Колбэк применяется к каждому элементу массива, а результат его работы записывается в новый массив.

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

numbers.map((number) => number * 2);
console.log(numbers);
// [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] — исходный массив не изменился
```

Метод умножил все элементы на 2, но не изменил исходный массив — просто результат нигде не сохранён. Чтобы увидеть результат работы `map()`, его нужно сохранить в новую переменную:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const newArray = numbers.map((number) => number * 2);

console.log(newArray);
// [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Два способа объявить функцию

- **function declaration** — функция объявляется через ключевое слово **function**:

```
function greet() {}
```

- **function expression** — функция объявляется через переменную, часто в виде стрелочной функции:

```
const calc = (number) => number * 2;
```

Условная логика внутри map()

Способ 1 — через **if / else**:

```
const newArrayBooks = arrayBooks.map((book, index) => {  
  if (index === 0) {  
    return { ...book, year: 2000 };  
  } else {  
    return { ...book, year: 2025 };  
  }  
});
```

Способ 2 — через тернарный оператор, с обращением к полям объекта через **...** (spread):

```
const newArrayBooks = arrayBooks.map((book, index) => {  
  return { ...book, year: index === 0 ? 2024 : 3000 };  
});
```

filter()

Не изменяет исходный массив — создаёт новый массив из элементов, которые прошли проверку по условию. Как и с **map()**, результат нужно сохранить в переменную.

```
const newBooks = arrayBooks.filter((book) => book.year > 2000);  
// массив книг с годом выпуска больше 2000
```

Работает так же и со строковыми условиями сравнения.

includes()

Проверяет, есть ли искомый элемент в массиве, и возвращает логическое значение (**true/false**). Работает только с примитивными значениями (числа, строки, **boolean** и т.д.) — для поиска объектов по условию нужен другой метод.

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
const hasThree = numbers.includes(3);
```

```
console.log(hasThree); // true
```

find()

Возвращает первый попавшийся элемент массива, который подходит под условие в колбэке (если подходящих несколько — вернётся только первый; если ни один не подошёл — undefined).

```
const foundBook = arrayBooks.find((book) => book.genre === "Научная фантастика");
```

reverse(), concat(), slice()

- `reverse()` — меняет порядок элементов исходного массива на обратный (это мутирующий метод, в отличие от остальных в этом разделе).
- `concat()` — соединяет два массива в новый, не изменяя исходные.
- `slice(начало, конец)` — принимает два индекса и "вырезает" часть массива между ними в новый массив, не трогая исходный.

```
const a = [1, 2, 3];
const b = [4, 5, 6];
const merged = a.concat(b);
// [1, 2, 3, 4, 5, 6] — новый массив, а и b не изменились

const numbers = [1, 2, 3, 4, 5];
const part = numbers.slice(1, 3);
// [2, 3] — элементы с индекса 1 до 3 (не включая индекс 3)
```